DOSSIER TECHNIQUE · PROJET PHP MVC

Touche pas au klaxon

Application de covoiturage pour les trajets inter-sites d’une entreprise.

Technologies : PHP 8.2+, architecture MVC, MySQL / MariaDB, Bootstrap 5, Sass

Livrable : dépôt GitHub du code source, schéma MCD, MLD, procédure d’installation, comptes de test

Étudiant : NOM / PRÉNOM À RENSEIGNER

Dépôt GitHub : LIEN À AJOUTER APRÈS PUBLICATION DU DÉPÔT

Document rédigé à partir du brief « Touche pas au klaxon » et des annexes de données et visuels fournies.

Sommaire

1. Mission et périmètre fonctionnel

2. Conception technique et architecture MVC

3. Conformité aux contraintes / sécurité

4. MCD : modèle conceptuel de données

5. MLD : modèle logique de données

6. Installation et exécution

7. Comptes de démonstration

8. Contrôles qualité et scénarios de test

9. Concordance avec les visuels fournis

10. Restitution et vérifications finales

Précision importante. Le brief demande le code PHP complet sur un dépôt GitHub. Ce PDF documente le projet fourni dans le ZIP, mais la publication du dépôt ne peut être effectuée qu'avec l'accès à votre compte GitHub. Remplacer le champ du lien après publication.

1. Mission et périmètre fonctionnel

Les salariés d’une entreprise multi-sites effectuent régulièrement des trajets similaires avec des véhicules peu occupés. « Touche pas au klaxon » vise à favoriser le covoiturage en rendant visibles les trajets programmés sur l’intranet.

Rôles utilisateurs

Rôle	Droits
Visiteur non connecté	Consulter uniquement les trajets futurs avec des places disponibles, triés chronologiquement ; accéder à la connexion.
Employé connecté	Consulter les détails de contact en modale ; proposer un trajet ; modifier et supprimer ses propres trajets.
Administrateur	Voir tous les trajets ; CRUD des trajets et agences ; liste des employés RH en consultation uniquement.

Règles de gestion

• Agence de départ différente de l’agence d’arrivée.

• Départ dans le futur lors de la création / modification ; date d’arrivée strictement postérieure au départ.

• 1 à 8 places totales ; de 0 au total de places disponibles.

• Aucun CRUD sur les employés, fournis en annexe via une extraction du SI RH.

• Après chaque écriture valide : redirection vers la liste et affichage d’un message flash.

• Accès prévu depuis les postes de travail internes ; aucune application mobile dédiée n’est demandée.

2. Conception technique et architecture MVC

Application PHP organisée selon le modèle MVC, avec point d’entrée unique public/index.php et routeur HTTP personnalisé app/Core/Router.php. La couche contrôleur gère les droits et les requêtes, les services appliquent les règles métier, les repositories exécutent les requêtes PDO préparées, et les vues PHP génèrent le HTML.

Couche	Fichiers principaux	Fonction
Routeur / cœur	Core/Router.php, Http.php, Auth.php	Routes GET/POST, sessions, CSRF, redirection 303, messages flash.
Contrôleurs	AuthController, TripController, AdminController	Connexion et contrôle d’accès, orchestration des pages.
Métier	Services/TripService.php, AgencyService.php, TripValidator.php	Validation, autorisation, CRUD, gestion erreurs métier.
Données	Repositories/*, Core/Database.php	PDO MySQL, requêtes préparées, séparation des écritures.
Vues	Views/*, partials/layout.php	Affichage HTML, modales et formulaires Bootstrap.
Style	resources/scss/*, public/assets/css/*	Thème Sass, variables Bootstrap, CSS prêt à l’emploi.

3. Conformité aux contraintes et sécurité

Technologies et réutilisabilité

L'application utilise PHP 8.2+, MVC, MySQL/MariaDB et un routeur HTTP. Les vues s'appuient sur Bootstrap (version locale fournie pour un intranet hors Internet) et sur des sources Sass paramétrant les variables de couleur. Les composants métier et repositories sont isolés pour favoriser leur réemploi sur les futurs sites thématiques.

Palette du brief : #f1f8fc (fond), #0074c7 (primaire), #00497c (bleu foncé), #384050 (gris), #cd2c2e (danger), #82b864 (succès).

Protection des données et des opérations

• Stockage des secrets en .env non versionné ; accès HTTP limité au dossier public/.

• Mots de passe hachés par password_hash(), comparés avec password_verify() ; régénération de session après connexion.

• Authentification obligatoire pour lire les coordonnées d'un conducteur ; permissions auteur ou administrateur pour modifier/supprimer.

• Accès administrateur exclusif pour la gestion des agences et la liste complète des employés.

• Jeton CSRF sur les requêtes POST, échappement HTML et requêtes SQL préparées pour réduire les risques courants.

• En production : serveur accessible uniquement depuis le LAN/VPN de l'entreprise, pare-feu, HTTPS, journalisation et rotation des mots de passe de démonstration.

Politique de contrôle de code

DocBlock sur les classes et méthodes métier, PHPStan (configuration niveau 5) et tests PHPUnit des opérations d'écriture. Les tests fonctionnels avec MySQL sont à exécuter après installation locale des dépendances.

4. MCD — modèle conceptuel de données

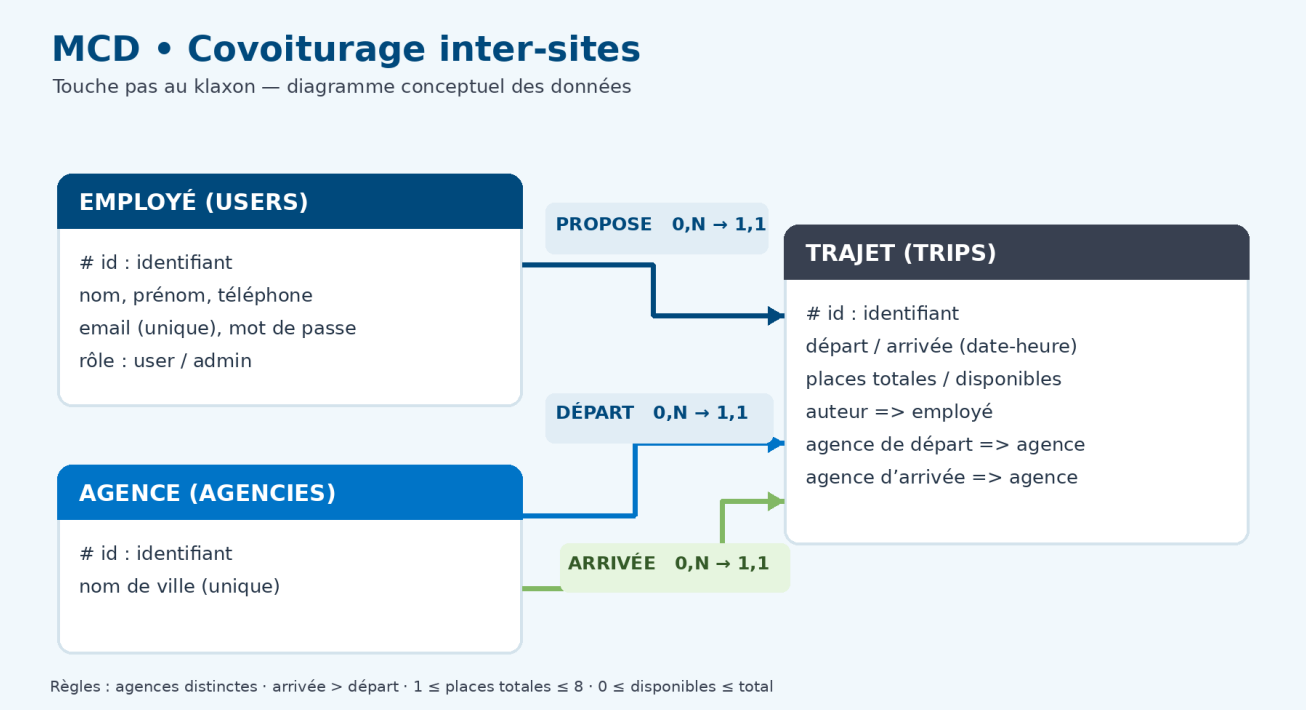


Figure 1 : entités, associations et cardinalités du modèle de covoiturage.

Un employé peut proposer zéro à plusieurs trajets (0,N), chaque trajet étant lié à un seul employé (1,1). Une agence peut être le départ ou l'arrivée de zéro à plusieurs trajets (0,N) ; chaque trajet possède exactement un départ et une arrivée (1,1).

5. MLD — modèle logique de données

Relations et contraintes

AGENCIES (#id, name UNIQUE)

USERS (#id, last_name, first_name, phone, email UNIQUE, password_hash, role, created_at)

TRIPS (#id, departure_at, arrival_at, seats_total, seats_available, created_at, updated_at, departure_agency_id → AGENCIES.id, arrival_agency_id → AGENCIES.id, owner_id → USERS.id)

Les clés primaires sont précédées de « # ». Les champs suivis de → sont des clés étrangères. Contraintes SQL : CHECK agences différentes, arrivée > départ, 1–8 places totales, places disponibles ≤ total ; suppression d'une agence référencée interdite.

Contenu initial fourni en annexe

Table	Alimentation / utilité
agencies	12 villes importées depuis agences.txt : Paris, Lyon, Marseille, Toulouse, Nice, Nantes, Strasbourg, Montpellier, Bordeaux, Lille, Rennes, Reims.
users	20 employés importés depuis users.txt, avec nom, prénom, téléphone, email. Un rôle administrateur est attribué à Alexandre Martin pour les essais.
trips	4 trajets fictifs ajoutés pour tester l'affichage (dont un trajet complet à exclure de l'accueil).

Scripts à exécuter

database/01_schema.sql crée la base et les tables ; database/02_seed.sql fournit les données initiales et les trajets de démonstration. Ces deux scripts doivent être présents dans le dépôt GitHub.

6. Installation et lancement

Préparer la base MySQL/MariaDB

1. Installer PHP 8.2+ avec l'extension pdo_mysql, MySQL/MariaDB et, facultativement, MySQL Workbench.

2. Dans MySQL Workbench, ouvrir puis exécuter database/01_schema.sql, ensuite database/02_seed.sql.

3. Copier .env.example en .env et compléter DB_HOST, DB_PORT, DB_NAME, DB_USER et DB_PASSWORD. Le nom attendu est touche_pas_au_klaxon.

Lancer le site localement

4. Depuis le dossier racine, exécuter :

php -S 127.0.0.1:8000 -t public public/router.php

5. Ouvrir http://127.0.0.1:8000. Le serveur de développement de PHP utilise le routeur dédié pour les URL propres. Pour Apache, définir DocumentRoot sur public/ avec réécriture activée.

Qualité et compilation des styles

Pour PHPUnit et PHPStan : composer install, puis composer test et composer analyse. Pour reconstruire Bootstrap à partir des variables Sass : npm install et npm run build:css. Le projet comporte déjà des fichiers Bootstrap et CSS utilisables sans compilation.

Exploitation intranet

Déployer derrière un reverse proxy interne (HTTPS), limiter l'accès aux employés via pare-feu/VPN, supprimer les identifiants de test avant l'ouverture, sauvegarder la base et activer les logs. L'application n'a pas vocation à être publiée sur un hébergement public.

7. Identifiants des comptes de démonstration

Les comptes ont été repris des données RH fournies. Les mots de passe ci-dessous ont été créés exclusivement pour le jeu d'essais et sont enregistrés sous forme de hash dans SQL.

Rôle	Identifiant (email)	Mot de passe
Administrateur	alexandre.martin@email.fr	Admin@2026!
Utilisateur	sophie.dubois@email.fr	User@2026!

Attention : ces identifiants ont une vocation exclusivement pédagogique. Les supprimer ou les remplacer avant tout déploiement réel. Le système RH n'a pas fourni de mots de passe dans l'annexe.

8. Tests et contrôle qualité

Tests automatisés prévus dans le dépôt

• TripServiceTest : création, modification par l'auteur, refus de suppression par un tiers, suppression par l'administrateur, rejet des agences identiques, rejet du dépassement des places.

• AgencyServiceTest : création, renommage, suppression, refus de doublon et refus de supprimer une agence utilisée.

• Tests unitaires basés sur des dépôts simulés en mémoire pour isoler les règles d'écriture ; les scripts SQL peuvent être validés avec MySQL lors des tests d'intégration.

Vérifications effectuées et restantes

Réalisées : analyse syntaxique de tous les fichiers PHP avec php -l ; scénarios rapides (smoke tests) des services de création, mise à jour, suppression et autorisation avec des dépôts simulés.

À faire sur le PC de rendu : exécuter PHPUnit, PHPStan, importer MySQL, réaliser un test manuel complet des écrans et publier le dépôt GitHub. Le conteneur utilisé pour générer les fichiers ne dispose pas de pdo_mysql/Composer : aucun test réel de connexion MySQL ne peut être revendiqué.

9. Concordance avec les visuels fournis

Les annexes montrent un tableau des trajets, la navigation adaptée au rôle de l'utilisateur, une modale de détails et un message flash après modification. Le développement reprend ces comportements avec une présentation bureautique Bootstrap.

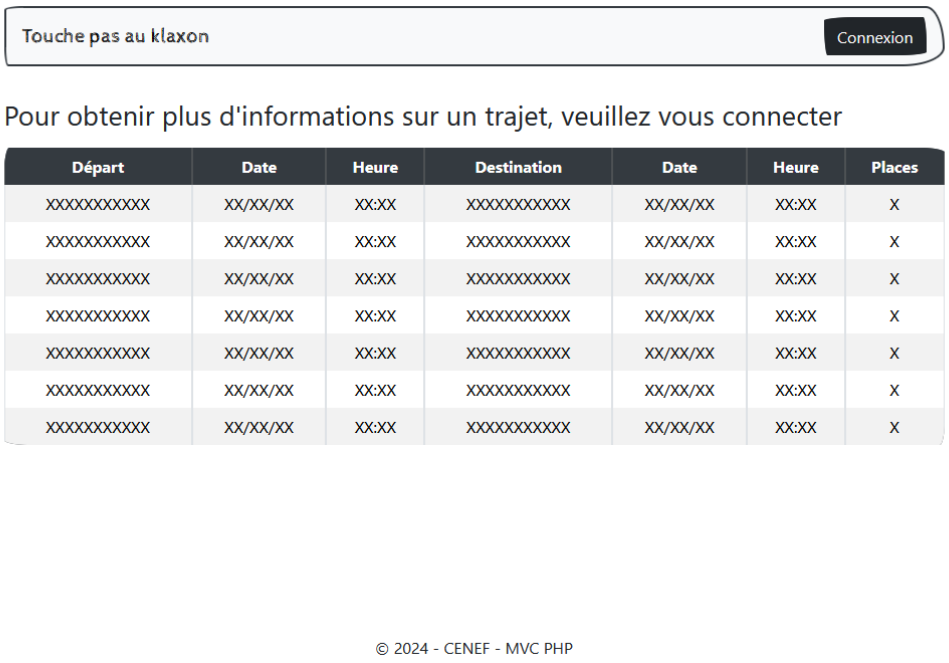


Figure 2. Référence fournie : visiteur non connecté, tableau filtré.

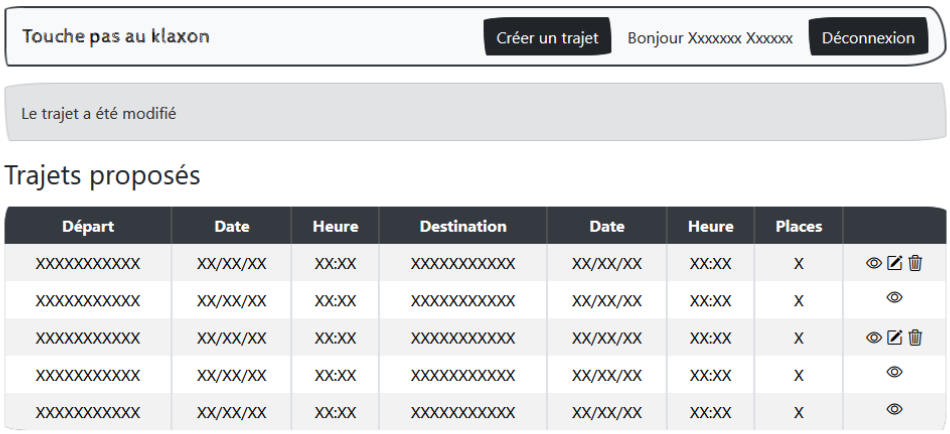


Figure 3. Référence fournie : confirmation d'édition et actions auteur.

10. Restitution et vérifications finales

Éléments livrés

• Code PHP en architecture MVC, vues Bootstrap, style Sass et routeur HTTP.

• Scripts SQL de création et d'alimentation depuis les annexes (20 employés, 12 agences).

• README.md décrivant installation, utilisation, droits, configuration et tests.

• Schéma MCD (docs/MCD.png) et MLD textuel (docs/MLD.txt).

• Tests unitaires PHPUnit et configuration d'analyse statique PHPStan.

• Comptes de démonstration administrateur et employé consignés ci-dessus.

Dépot GitHub

URL du dépôt : À renseigner dès que le code est publié sur votre compte GitHub. Le ZIP fourni est prêt pour cette publication, mais aucune publication n'a été faite à votre place.

Liste de contrôle avant le rendu

[ ] Compléter le nom et prénom sur la couverture et renommer le PDF conformément aux consignes.

[ ] Créer un dépôt GitHub et y pousser le code, les scripts SQL et README.md ; ajouter le lien réel dans le PDF.

[ ] Tester l'installation locale avec pdo_mysql + MySQL Workbench.

[ ] Installer Composer, exécuter PHPUnit / PHPStan et corriger les éventuels avertissements.

[ ] Vérifier les deux comptes de démonstration et toutes les opérations CRUD/flash.